

JAXMg: A multi-GPU linear solver in JAX

Jacob Tutt^{1,2*} and Roeland Wiersema^{3*}

¹ Cavendish Astrophysics, University of Cambridge, Cambridge CB3 0HE, UK ² Kavli Institute for Cosmology, University of Cambridge, Cambridge CB3 0HA, UK ³ Center for Computational Quantum Physics, Flatiron Institute, 162 Fifth Avenue, New York, NY 10010, USA ^{*} Corresponding author * These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Prashant Jha](#)

Submitted: 03 September 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Solving large dense linear systems and eigenvalue problems is a core requirement across scientific computing, but scaling these operations beyond a single GPU remains challenging within modern programming frameworks. Highly optimized multi-GPU solver libraries exist, yet they are hard to integrate into composable, just-in-time (JIT) compiled Python workflows.

JAXMg provides distributed dense linear algebra for JAX, enabling linear solves and decompositions for matrices that exceed single-GPU memory limits. By interfacing JAX with NVIDIA's cuSOLVERM through an XLA Foreign Function Interface, JAXMg exposes distributed GPU routines as JIT-compatible JAX primitives. This design allows scalable linear algebra to be embedded directly within JAX programs, preserving composability with JAX transformations and enabling multi-GPU and multi-node execution in end-to-end scientific workflows.

Statement of need

GPUs now supply a large fraction of the floating-point throughput of both supercomputers and smaller multi-GPU workstations, and dense linear algebra remains a critical building block of many numerical methods.

JAX ([Bradbury et al., 2018](#)) has become a widely adopted framework for scientific computing because it combines a simple user experience with JIT compilation and automatic differentiation. The JAX ecosystem has expanded rapidly, with libraries for neural networks ([Heek et al., 2024](#)), Bayesian inference ([Cabezas et al., 2024](#)), differential equations ([Kidger, 2021](#)), variational Monte Carlo ([Vicentini et al., 2022](#)), and full physics simulation environments ([Freeman et al., 2021](#)). These workflows frequently solve linear systems or compute eigendecompositions, either inside a larger simulation loop or inside differentiable optimization, and matrix size often limits the scale of feasible simulations.

Despite this growth, the ecosystem still lacks distributed dense linear solver routines that scale across multiple GPUs while remaining usable from idiomatic JAX programs. A dedicated backend package makes this integration reusable across scientific applications while containing the native CUDA dependencies and redistribution machinery in one implementation. JAXMg therefore complements existing solver libraries and JAX application frameworks by providing the execution and memory-management layer needed to connect them.

State of the field

Within the JAX ecosystem, Lineax (Rader et al., 2023) provides composable linear-operator abstractions and a range of direct and iterative solvers. However, its dense factorizations do not extend across multiple GPUs and therefore remain limited by single-device memory. Scaling these operations beyond a single device consequently requires turning to distributed dense linear-algebra libraries outside the JAX ecosystem.

ScaLAPACK (Blackford et al., 1997) provides distributed factorizations for CPU clusters, SLATE (Gates et al., 2019) targets distributed systems with hardware accelerators, and MAGMA (Abdelfattah et al., 2024) provides algorithms for heterogeneous CPU-GPU systems. NVIDIA's cuSOLVERMp (Corporation, n.d.) supplies distributed dense solvers for multi-GPU and multi-node execution. However, their native C, C++, or Fortran interfaces, together with their execution models and data-layout requirements, complicate integration into Python-based scientific workflows.

JAXMg bridges these approaches by combining idiomatic, JIT-compatible JAX integration with scalable multi-GPU and multi-node dense solvers.

Software design

JAXMg connects JAX to NVIDIA's distributed dense linear algebra library cuSOLVERMp (Corporation, n.d.) via an XLA Foreign Function Interface (FFI) C++/CUDA extension. The supported cuSOLVERMp routines support the JAX dtypes float32, float64, complex64, and complex128, with CUDA 12 and CUDA 13 backends available for both x86_64 and aarch64 systems.

Parallelized linear algebra algorithms, like the ones implemented by cuSOLVERMp, require a distributed data layout to ensure proper load balancing of the available computational power (Dongarra et al., 1994). For JAXMg, the central challenge is constructing this layout in a fast and memory efficient manner. JAXMg transforms the matrix buffers that are donated through the FFI in place and reuses a single bounded scratch allocation across all stages. JAXMg performs the required data redistribution in three stages.

Local memory-layout conversion

The first stage reconciles the physical memory layouts used by JAX and cuSOLVERMp. JAX stores each local matrix shard in row-major order, whereas cuSOLVERMp requires column-major local buffers. While XLA can materialize a column-major FFI input, doing so requires a second full-sized local matrix allocation, defeating the low-memory design. JAXMg instead applies a parallel implementation of the rectangular permutation decomposition introduced by Catanzaro et al. (2014) directly to each donated buffer. The method expresses the layout change as modular column and row permutations, processed in batches bounded by the shared scratch allocation. The logical matrix remains unchanged and, because the conversion is entirely local, this stage runs concurrently on every GPU without inter-device communication.

Edge-padding alignment

Due to the 2D block-cyclic layout required by the cuSOLVERMp backend, JAXMg pads a local JAX shard before it enters the native backend if either dimension is not divisible by the corresponding tile dimension T_A . This provides enough local capacity for the solver layout, but leaves padding between neighbouring shards in the global process grid. As a result, the destination of a solver tile may still contain part of another tile, so the block-cyclic redistribution described below cannot yet move complete tile slabs directly. JAXMg therefore compacts the logical matrix towards the global top-left, leaving the padding on the global right and bottom edges, as illustrated in Figure 1.

83 The compaction proceeds in two passes. Column slabs are first shifted left within each
84 process row, after which row slabs are shifted upwards within each process column. Since the
85 padding provides empty destinations, these movements form open chains and do not require an
86 additional temporary buffer for preserving overwritten data. Dependencies between movements
87 prevent an entire pass from being executed at once, so each pass is divided into ordered
88 waves. Within each wave, the largest slabs that fit in the shared scratch allocation are moved
89 concurrently across independent process rows or columns.

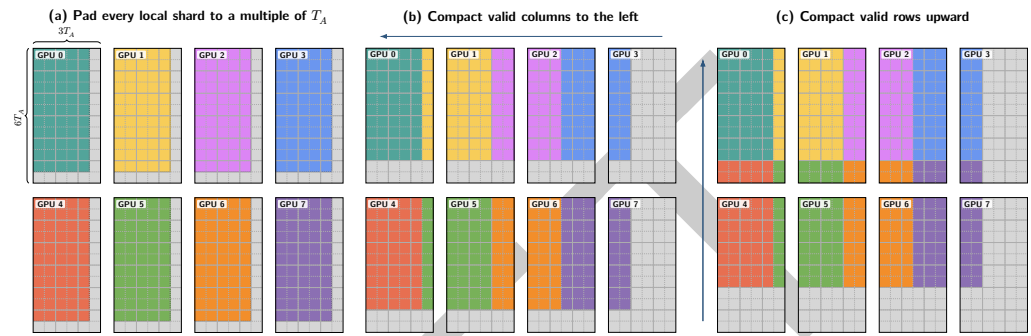


Figure 1: Demonstration of the edge-padding alignment across a 2×4 GPU process grid. (a) Initial JAX block-sharded layout after each local shard is padded to a multiple of T_A . (b) Horizontal compaction shifts valid column slabs to the left within each process row. (c) Vertical compaction then shifts valid row slabs upwards within each process column, leaving all padding on the global right and bottom edges. Colours indicate the GPU that originally owned each matrix entry, while grey cells denote padding.

90 Unlike the in-place redistribution handled by the native backend, this initial capacity padding
91 must be performed by JAX because an existing donated allocation cannot be expanded once
92 assigned. Materializing the padded array thus temporarily requires both the original and padded
93 buffers, reducing the matrix size that fits in available GPU memory. Padding should therefore
94 be avoided where possible by choosing a tile size that divides both dimensions of every local
95 matrix shard.

96 2D block-cyclic redistribution

97 Finally, JAXMg constructs the 2D block-cyclic layout required by cuSOLVERMp. For a process
98 grid with P_r rows and P_c columns, tiles are distributed in round-robin order over both axes, such
99 that the tile at global tile coordinate (i, j) is assigned to owner $(i, j) = (i \bmod P_r, j \bmod P_c)$.

100 JAXMg constructs an explicit mapping from every source tile to its destination and applies
101 it in two separable phases. First, the column-owner mapping is decomposed into disjoint
102 permutation cycles that rotate complete tile-column slabs within each process row. The
103 corresponding cycles run concurrently across all process rows until every tile has the correct
104 process-column owner. The same procedure is then applied at the row level within each process
105 column and parallelized across the process columns, as shown in Figure 2.

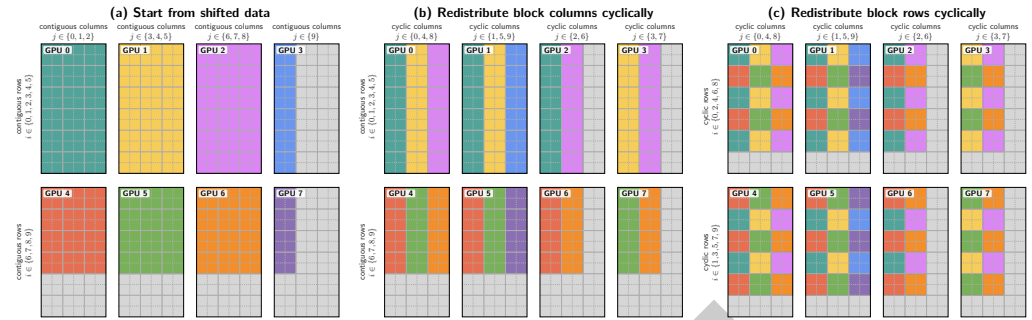


Figure 2: Demonstration of the redistribution from the compacted JAX block-sharded layout to the 2D block-cyclic layout required by cuSOLVERMp across a 2×4 GPU process grid. (a) The top-left-aligned matrix is partitioned into $T_A \times T_A$ tiles. (b) Tile-column slabs are redistributed cyclically within each process row, assigning global tile column j to process column $j \bmod P_c$. (c) Tile-row slabs are subsequently redistributed within each process column, assigning global tile row i to process row $i \bmod P_r$. Colours indicate the original GPU ownership of each matrix entry, while grey cells denote padding.

During each cycle, a slab is packed into the send scratch slot, transferred into the receive scratch slot, and unpacked into its destination, while a third saved slot preserves data that would otherwise be overwritten before it is forwarded. This results in a highly memory efficient implementation of the memory remapping.

Solver orchestration

An important design feature of JAXMg is that this entire pipeline is performed within a single fused C++/CUDA FFI call that the user never has to interact with. This design enables writing complex, JIT-compatible JAX programs while delegating the computationally intensive components to a compiled backend. Simply pass JAXMg an ordinary JAX array sharded over a two-dimensional device mesh. The native backend handles the local memory-layout conversion, 2D block-cyclic redistribution, distributed solver execution, and restoration of the result to its original JAX layout.

Research impact statement

JAXMg is integrated into NetKet (Vicentini et al., 2022), one of the most widely used open-source frameworks for variational Monte Carlo, where it backs the distributed linear solve at the heart of stochastic reconfiguration (Sorella, 1998). Distributing it removes the ceiling on how large an ansatz NetKet users can optimize. Additionally, JAXMg produced the time-dependent variational Monte Carlo (Carleo & Troyer, 2017; Schmitt & Heyl, 2020) results of Wan et al. (2026). A future release of jVMC (Schmitt & Reh, 2022), will also feature support for JAXMg.

JAXMg is also being used to scale BayesEoR (Tutt et al., in prep.), a Bayesian forward-modelling framework for 21-cm power-spectrum recovery with next-generation interferometric experiments (Bull et al., 2026; Sims et al., 2024). As identified by Burba et al. (2023), unbiased power-spectrum recovery requires the uncertainty associated with wide-field foreground effects to be fully propagated, rapidly inflating the nuisance parameterisation to $\mathcal{O}(10^6)$ coefficients. JAXMg's distributed routines enable the construction of a statistically optimal reduced KL basis of $\mathcal{O}(10^3)$ modes (Shaw et al., 2014) and, through analytical marginalisation of the likelihood, collapse the inference to just $\mathcal{O}(10^1)$ sampled power-spectrum parameters. Access to the Bayesian evidence through integration with nested sampling (Skilling, 2006; Yallup et al., 2026) thus enables robust statistical validation through model comparison, which is vital in the low-signal-to-noise, highly degenerate regime characteristic of 21-cm cosmology.

In a series of benchmark experiments, we also evaluate the power of JAXMg scalability by investigating the currently implemented routines across a large number of GPUs. As a highlight, we perform a successful Cholesky solve of a float32 matrix of size $1.5 \times 10^6 \times 1.5 \times 10^6$ across 64 NVIDIA H200s in approximately 11 minutes (Tutt & Wiersema, 2026).

We envision future scientific applications in areas such as Bayesian inference (Cabezas et al., 2024), Gaussian processes (Pinder & Dodd, 2022), tensor networks (Bañuls, 2023; Schollwöck, 2011), and computational electromagnetics (Gueuning et al., 2026; Harrington, 1993).

AI usage disclosure

Claude Code was used during software development for code exploration and debugging. Large language models were used to assist with language polishing. All code and text was reviewed by Humans.

Acknowledgements

We want to thank Dennis Bollweg, Alex Chavin, Geraud Krawezik, Dylan Simon and Nils Wentzell for their help with code development. We also want to acknowledge the help of Ao Chen and Riccardo Rende with testing the code in applied settings. RW is grateful to Simon Tartakovsky for his suggestions on the 1D cyclic algorithm. Finally, we want to thank Filippo Vicentini for his suggestions on code distribution. RW acknowledges support from the Flatiron Institute. The Flatiron Institute is a division of the Simons Foundation. JT is supported by the Harding Distinguished Postgraduate Scholars Programme (HDPSP) and the Science and Technology Facilities Council (STFC) DTP Studentship.

The authors acknowledge the use of resources provided by the Isambard-AI National AI Research Resource (AIRR). Isambard-AI (McIntosh-Smith et al., 2024) is operated by the University of Bristol and is funded by the UK Government's Department for Science, Innovation and Technology (DSIT) via UK Research and Innovation; and the Science and Technology Facilities Council [ST/AIRR/I-A-I/1023].

References

- Abdelfattah, A., Beams, N., Carson, R., Ghysels, P., Kolev, T., Stitt, T., Vargas, A., Tomov, S., & Dongarra, J. (2024). MAGMA: Enabling exascale performance with accelerated BLAS and LAPACK for diverse GPU architectures. *The International Journal of High Performance Computing Applications*, 38(5), 468–490. <https://doi.org/10.1177/10943420241261960>
- Bañuls, M. C. (2023). Tensor network algorithms: A route map. *Annual Review of Condensed Matter Physics*, 14(1), 173–191. <https://doi.org/10.1146/annurev-conmatphys-040721-022705>
- Blackford, L. S., Choi, J., Cleary, A., D'Azevedo, E., Demmel, J., Dhillon, I., Dongarra, J., Hammarling, S., Henry, G., Petitet, A., & others. (1997). *ScaLAPACK users' guide*. SIAM. <https://doi.org/10.1137/1.9780898719642>
- Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.3.13). <http://github.com/google/jax>
- Bull, P., Burba, J., Ceccotti, E., Chakraborty, A., Choudhuri, S., Datta, A., Elahi, K. M. A., Giri, S. K., Ito, T., Jelic, V., Mao, Y., Mertens, F., Munshi, S., Nunhokee, R., Offringa, A. R., Pal, S. K., Sagar, R., Shan, H., Sims, P. H., ... Zhu, Z. (2026). *Foreground*

- 179 *Characterization and Mitigation in the Observations of the CD/EoR with the SKA.* <https://arxiv.org/abs/2607.01048>
180
- 181 Burba, J., Sims, P. H., & Pober, J. C. (2023). All-sky modelling requirements for Bayesian 21
182 cm power spectrum estimation with bayeseor. *Monthly Notices of the Royal Astronomical*
183 *Society*, 520(3), 4443–4455. <https://doi.org/10.1093/mnras/stad401>
- 184 Cabezas, A., Corenflos, A., Lao, J., & Louf, R. (2024). *BlackJAX: Composable Bayesian*
185 *inference in JAX.* <https://doi.org/10.48550/arXiv.2402.10797>
- 186 Carleo, G., & Troyer, M. (2017). Solving the quantum many-body problem with artificial
187 neural networks. *Science*, 355(6325), 602–606. <https://doi.org/10.1126/science.aag2302>
- 188 Catanzaro, B., Keller, A., & Garland, M. (2014). A decomposition for in-place matrix
189 transposition. *Proceedings of the 19th ACM SIGPLAN Symposium on Principles and*
190 *Practice of Parallel Programming*, 193–206. <https://doi.org/10.1145/2555243.2555253>
- 191 Corporation, N. (n.d.). *cuSOLVERMp API.* <https://docs.nvidia.com/cuda/cusolvermp/index.html>
192
- 193 Dongarra, J. J., Vandegeijn, R. A., & Walker, D. W. (1994). Scalability Issues Affecting the
194 Design of a Dense Linear Algebra Library. *Journal of Parallel and Distributed Computing*,
195 22(3), 523–537. <https://doi.org/10.1006/jpdc.1994.1108>
- 196 Freeman, C. D., Frey, E., Raichuk, A., Girgin, S., Mordatch, I., & Bachem, O. (2021). *Brax -*
197 *A Differentiable Physics Engine for Large Scale Rigid Body Simulation* (Version 0.14.0).
198 <https://doi.org/10.48550/arXiv.2106.13281>
- 199 Gates, M., Kurzak, J., Charara, A., YarKhan, A., & Dongarra, J. (2019). SLATE: Design of a
200 modern distributed and accelerated linear algebra library. *Proceedings of the International*
201 *Conference for High Performance Computing, Networking, Storage and Analysis*, 1–18.
202 <https://doi.org/10.1145/3295500.3356223>
- 203 Gueuning, Q., Lera Acedo, E. de, Brown, A. K., & Fagnoni, N. (2026). *A Fast Direct Solver*
204 *for Mutual Coupling Analysis of Large Arrays of Reflector Antennas.* <https://doi.org/10.48550/arXiv.2604.10239>
205
- 206 Harrington, R. F. (1993). *Field Computation by Moment Methods.* Wiley–IEEE Press.
207 <https://doi.org/10.1109/9780470544631>
- 208 Heek, J., Levskaya, A., Oliver, A., Ritter, M., Rondepierre, B., Steiner, A., & Zee, M.
209 van. (2024). *Flax: A neural network library and ecosystem for JAX* (Version 0.12.2).
210 <http://github.com/google/flax>
- 211 Kidger, P. (2021). *On Neural Differential Equations* [PhD thesis, University of Oxford].
212 <https://doi.org/10.48550/arXiv.2202.02435>
- 213 McIntosh-Smith, S., Alam, S. R., & Woods, C. J. (2024). Isambard-AI: A leadership-class
214 supercomputer optimised specifically for artificial intelligence. *CUG 2024 Proceedings*,
215 44–54. <https://doi.org/10.1145/3725789.3725794>
- 216 Pinder, T., & Dodd, D. (2022). GPJax: A Gaussian Process Framework in JAX. *Journal of*
217 *Open Source Software*, 7(75), 4455. <https://doi.org/10.21105/joss.04455>
- 218 Rader, J., Lyons, T., & Kidger, P. (2023). Lineax: Unified linear solves and linear least-squares
219 in JAX and equinox. *AI for Science Workshop at Neural Information Processing Systems.*
220 <https://doi.org/10.48550/arXiv.2311.17283>
- 221 Schmitt, M., & Heyl, M. (2020). Quantum many-body dynamics in two dimensions with
222 artificial neural networks. *Physical Review Letters*, 125(10), 100503. <https://doi.org/10.1103/PhysRevLett.125.100503>
223
- 224 Schmitt, M., & Reh, M. (2022). jVMC: Versatile and performant variational Monte Carlo

- 225 leveraging automated differentiation and GPU acceleration. *SciPost Phys. Codebases*, 2,
226 <https://doi.org/10.21468/SciPostPhysCodeb.2>
- 227 Schollwöck, U. (2011). The density-matrix renormalization group in the age of matrix product
228 states. *Annals of Physics*, 326(1), 96–192. <https://doi.org/10.1016/j.aop.2010.09.012>
- 229 Shaw, J. R., Sigurdson, K., Pen, U.-L., Stebbins, A., & Sitwell, M. (2014). All-Sky
230 Interferometry with Spherical Harmonic Transit Telescopes. *The Astrophysical Journal*,
231 781(2), 57. <https://doi.org/10.1088/0004-637X/781/2/57>
- 232 Sims, P. H., Burba, J., & Pober, J. C. (2024). BayesEoR: Bayesian 21-cm Power Spectrum
233 Estimation from Interferometric Visibilities. *Journal of Open Source Software*, 9(103),
234 6667. <https://doi.org/10.21105/joss.06667>
- 235 Skilling, J. (2006). Nested sampling for general Bayesian computation. *Bayesian Analysis*,
236 1(4), 833–859. <https://doi.org/10.1214/06-BA127>
- 237 Sorella, S. (1998). Green function monte carlo with stochastic reconfiguration. *Physical Review*
238 *Letters*, 80(20), 4558. <https://doi.org/10.1103/physrevlett.80.4558>
- 239 Tutt, J., & Wiersema, R. (2026). *JAXMg: Benchmarks*. [https://github.com/therooler/](https://github.com/therooler/jaxmg_benchmark)
240 [jaxmg_benchmark](https://github.com/therooler/jaxmg_benchmark)
- 241 Vicentini, F., Hofmann, D., Szabó, A., Wu, D., Roth, C., Giuliani, C., Pescia, G., Nys,
242 J., Vargas-Calderón, V., Astrakhantsev, N., & Carleo, G. (2022). NetKet 3: Machine
243 Learning Toolbox for Many-Body Quantum Systems. *SciPost Phys. Codebases*, 7. <https://doi.org/10.21468/SciPostPhysCodeb.7>
- 244
- 245 Wan, Z.-Q., Wiersema, R., & Zhang, S. (2026). Removing nodal and support-mismatch
246 pathologies in variational monte carlo via blurred sampling. *Phys. Rev. X*, 16, 031059.
247 <https://doi.org/10.1103/jrn5-gv19>
- 248 Wiersema, R. (2026). *Numerical simulation of D-Wave's quantum advantage experiment with*
249 *time-dependent variational Monte Carlo*. <https://doi.org/10.48550/arXiv.2609.01719>
- 250 Yallup, D., Kroupa, N., & Handley, W. (2026). *Nested Slice Sampling: Vectorized Nested*
251 *Sampling for GPU-Accelerated Inference*. <https://arxiv.org/abs/2601.23252>